

Chapter 7

Introduction to React

- React is an open-source JavaScript library developed by Facebook.
- Used to build interactive and fast web applications.
- React mainly handles the **View layer** in MVC architecture.
- Popular applications using React:
 - Facebook
 - Instagram
 - Netflix
 - WhatsApp Web

Advantages of React

- Component-based architecture
- Reusable code
- Faster rendering
- Easy maintenance
- Declarative programming
- Efficient DOM updates

Virtual DOM

Virtual DOM is a lightweight copy of the Real DOM.

Working:

1. React creates Virtual DOM
2. Compares old and new Virtual DOM
3. Updates only changed elements in Real DOM

Advantages:

- Faster UI updates
- Better performance
- Smooth rendering

SPA (Single Page Application)

SPA loads a single page and updates content without full page reload.

Features:

- Faster navigation
- Better user experience
- Uses react-router-dom
- Uses <Link> instead of <a> tag

Installation & Setup using Vite

Check Node.js

```
node -v  
npm -v
```

Create React App using Vite

1. Create Vite Project

```
npm create vite@latest
```

Starts the Vite setup wizard.

2. Enter Project Name

Example:

```
myapp
```

Creates a folder named myapp.

```
> npx  
> create-vite  
  
◆ Project name:  
  vite-project
```

3. Select Framework

Choose:

```
React
```

Used to create a React application.

```
◆ Project name:  
  myapp  
  
◆ Select a framework:  
  • Vanilla  
  ○ Vue  
  ○ React  
  ○ Preact  
  ○ lit  
  ○ Svelte  
  ○ Solid  
  ○ Ember  
  ○ Qwik  
  ○ Angular  
  ○ Marko  
  ○ Others
```

4. Select Variant

Choose:

```
JavaScript
```

- Easy for beginners

- Most tutorials use this

```
◇ Project name:
  myapp

◇ Select a framework:
  React

◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + React Compiler
  ● JavaScript
  ○ JavaScript + React Compiler
  ○ RSC
  ○ React Router v7 ↗ https://reactrouter.com
  ○ TanStack Router ↗ https://tanstack.com/router
  ○ RedwoodSDK ↗ https://rwsdk.com
  ○ Vike ↗ https://vike.dev
```

JavaScript + React Compiler is an advanced optimized version.

5. Vite asks to start

Install with npm and start now?

If you choose:

Yes

Vite automatically runs:

npm install

npm run dev

So the development server starts automatically.

```
◇ Project name:
  myapp

◇ Select a framework:
  React

◇ Select a variant:
  JavaScript

◆ Install with npm and start now?
  ● Yes / ○ No
```

6. Open in Browser

<http://localhost:5173/>

Default Vite development URL.

If port 5173 is already in use, Vite automatically uses another port like:

5174

5175

Example:

Port 5173 is in use, trying another one...

Shortcut Command

npm create vite@latest myapp -- --template react

Creates a React project directly without asking questions.

Part	Meaning
Npm	Node package manager
Create	Create a new project
vite@latest	Use latest Vite
Myapp	Project folder name
--	Pass options to Vite
--template react	Create a React project

Useful Vite Commands

Install and Start Project

cd project-name

npm install

npm run dev

- npm install → installs packages
- npm run dev → starts Vite server

Run on Different Port

npm run dev -- --port 3000

Stop Server

Ctrl + C

Build Project

```
npm run build
```

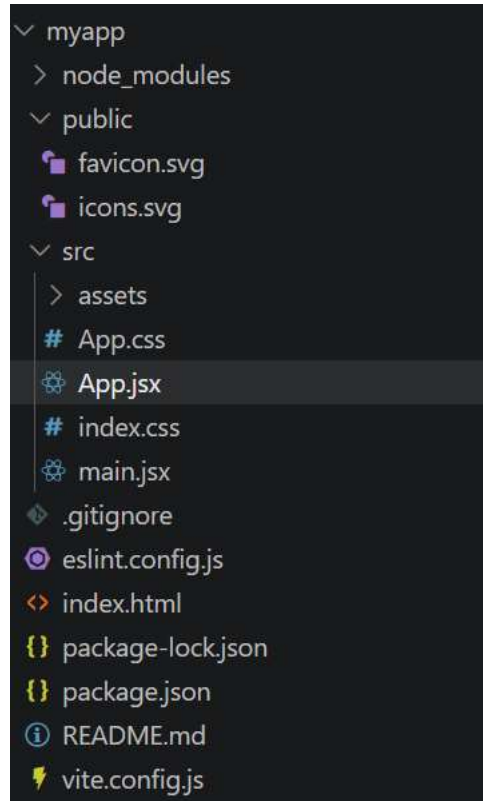
Creates production files.

Preview Build

```
npm run preview
```

Preview production version locally.

Folder Structure



File / Folder	What it Contains	Purpose / Use	Important Notes
node_modules/	All installed npm packages	React, Vite and other libraries are stored here	Created automatically after npm install. Very large folder. Never edit manually.
public/	Static files	Stores files that should be directly accessible in browser	Files are not processed by Vite. Used for favicon, PDFs, static images, etc.
src/	Main React code	Contains all components, CSS, logic and pages	Most project development happens here.
src/assets/	Images, icons, fonts, videos	Used to store component-related media files	Best place for images used inside React components.
src/App.jsx	Main React component	First component displayed in browser	Main UI design is usually written here.

File / Folder	What it Contains	Purpose / Use	Important Notes
src/main.jsx	Entry point file	Starts React app and connects with index.html	Flow: index.html → main.jsx → App.jsx
src/App.css	CSS for App component	Styles only App component UI	Used for component-specific styling.
src/index.css	Global CSS	Applies styles to whole application	Used for body styling, fonts, reset CSS, etc.
.gitignore	Ignored files list	Prevents unnecessary files from uploading to GitHub	Usually ignores node_modules, build files, environment files.
eslint.config.js	ESLint rules	Checks coding mistakes and code quality	Helps maintain clean and error-free code.
index.html	Main HTML page	Base HTML file loaded in browser	Contains <div id="root"></div> where React app renders.
package.json	Project configuration	Stores dependencies, scripts and project details	Contains commands like npm run dev, installed package names, versions.
package-lock.json	Exact dependency versions	Locks package versions for consistency	Automatically generated by npm.
README.md	Documentation file	Contains setup instructions and project notes	Written in Markdown format.
vite.config.js	Vite configuration	Used to customize Vite settings	Used for custom port, aliases, plugins, etc.

Rendering Flow

index.html → main.jsx → App.jsx

- index.html contains root div
- main.jsx connects React with browser
- App.jsx displays actual UI

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>myapp</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

It is the main HTML file of the React application.

When the application starts, this file loads first in the browser.

Code	Purpose	Explanation
<!doctype html>	Defines HTML version	Tells browser to use HTML5.
<html lang="en">	Root HTML tag	lang="en" specifies webpage language as English.
<head>	Metadata section	Contains title, favicon, charset and other settings.
<meta charset="UTF-8" />	Character encoding	Supports special characters, symbols and multiple languages.
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />	Website icon	Sets favicon shown in browser tab. File comes from public/ folder.
<meta name="viewport" content="width=device-width, initial-scale=1.0" />	Responsive design	Makes website responsive on mobile, tablet and desktop screens.
<title>myapp</title>	Browser tab title	Displays myapp on browser tab.
<body>	Visible webpage section	Contains content displayed in browser.
<div id="root"></div>	Root container	React app renders inside this div.

Code	Purpose	Explanation
<code><script type="module" src="/src/main.jsx"></script></code>	Loads React app	Starts application from main.jsx. type="module" enables modern JavaScript modules.

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Code	Purpose	Explanation
<code>import { StrictMode } from 'react'</code>	Imports StrictMode	Helps detect warnings and potential problems during development.
<code>import { createRoot } from 'react-dom/client'</code>	Imports createRoot	Used to render React app into browser DOM.
<code>import './index.css'</code>	Imports global CSS	Applies CSS styles to entire application.
<code>import App from './App.jsx'</code>	Imports App component	Loads the main React component.
<code>document.getElementById('root')</code>	Selects root div	Finds <code><div id="root"></div></code> from index.html.
<code>createRoot(...)</code>	Creates React root	Initializes React rendering system.

Code	Purpose	Explanation
<code>.render()</code>	Renders React app	Displays React components in browser.
<code><StrictMode></code>	Development checking tool	Shows warnings for unsafe or outdated React code.
<code><App /></code>	Main component	Main UI component rendered inside root div.

App.jsx

Code	Purpose	Explanation
import { useState } from 'react'	Imports Hook	useState is used to create and manage state in React.
import reactLogo from './assets/react.svg'	Imports image	Loads React logo from assets folder.
import viteLogo from './assets/vite.svg'	Imports image	Loads Vite logo from assets folder.
import heroImg from './assets/hero.png'	Imports image	Loads hero image from assets folder.
import './App.css'	Imports CSS	Applies styles for App.jsx.
function App()	React component	Main functional component of application.
const [count, setCount] = useState(0)	State creation	Creates state variable count with initial value 0.
Count	State variable	Stores current counter value.
setCount()	State update function	Updates count value.
return()	Returns JSX	Displays UI elements in browser.
<> </>	Fragment	Groups multiple elements without adding extra HTML tags.
<section id="center">	Section container	Main content section of page.
	Displays image	Shows imported image in browser.
className="hero"	CSS class	Applies CSS styling. React uses className instead of class.
Get started	Heading	Displays main heading.
<code>	Code formatting	Displays text in code style.
<button>	Button element	Creates clickable button.
onClick={() => setCount(...)}	Click event	Runs function when button is clicked.
setCount((count) => count + 1)	Updates state	Increases counter value by 1.

Code	Purpose	Explanation
Count is {count}	Dynamic value	Displays updated counter value using JSX expression.
	Hyperlink	Opens Vite documentation website.
target="_blank"	Open in new tab	Opens link in new browser tab.
<svg>	SVG icon	Displays vector icons.
export default App	Export component	Makes App component available for use in main.jsx.

Note:

React components can be created using both .js and .jsx extensions. .jsx is recommended for component files because it clearly indicates the file contains JSX (React UI code).

Extension	Common Usage
.jsx	React components
.js	JavaScript logic/helper files

Why Component Names Start with Capital Letters?

- React uses capitalization to identify components.
- Capital letters help React distinguish components from normal HTML elements.
- It is recommended to create component file names starting with capital letters to avoid confusion.

Name	React Treats It As
Header	React component
header	HTML tag

Correct:

```
function Header() {}
```

Wrong:

```
function header() {}
```

Understanding package.json

package.json is the main configuration file of a React project.

It contains:

- project information
- dependencies/packages
- scripts/commands
- package versions

Example:

```
{
  "name": "myapp",
  "version": "0.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  }
}
```

Important Fields in package.json

Field	Purpose
name	Project name
version	Project version
scripts	Commands like npm run dev
dependencies	Installed packages
devDependencies	Development packages

React Rendering

React renders UI using:

- createRoot()
- render()

Example

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
createRoot(document.getElementById('root')).render(
  <App />
)
```

React Components

- A React component is a JavaScript function that returns JSX (UI code).
- Components are reusable UI elements in a React application.
- Every part of UI in React is treated as a component.
- Component names should start with a capital letter.
- Components return JSX markup.

Default Component: App.jsx

When a React project is created using Vite, App.jsx is created by default. It is the main component rendered in browser.

Rendering Flow

index.html → main.jsx → App.jsx

Replace Default Code in App.jsx

```
function App() {  
  return (  
    <div>  
      <h1>Hello World!</h1>  
    </div>  
  )  
}  
export default App
```

Code	Purpose
function App()	Creates React component
return()	Returns JSX/UI
Hello World!	Displays heading in browser
export default App	Exports component

Create Your Own Component

Instead of modifying App.jsx, you can create your own component file.

Example:

src/Myapp.jsx

```
function Myapp() {  
  return (  
    <div>  
      <h1>Hello World!</h1>  
    </div>  
  )  
}  
export default Myapp
```

Import Component in main.jsx

Replace:

```
import App from './App.jsx'
```

with:

```
import Myapp from './Myapp.jsx'
```

Replace:

```
<App />
```

with:

```
<Myapp />
```

Updated main.jsx

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import Myapp from './Myapp.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <Myapp />  
  </StrictMode>,  
)
```

Now Myapp.jsx will render in browser instead of App.jsx.

JSX (JavaScript XML)

- JSX is a syntax extension of JavaScript used in React.
- It allows writing HTML-like code inside JavaScript.
- JSX makes UI code easier to read and write.
- React components return JSX to display UI in browser.
- JSX looks like HTML but follows stricter rules.

Example of JSX

```
function App() {  
  return <h1>Hello React</h1>  
}
```

JSX Rules

1. Return Single Root Element

A component must return only one parent element.

Wrong JSX

```
function App() {  
  return (  
    <h1>Hello</h1>  
    <p>React</p>  
  )  
}
```

Error

Adjacent JSX elements must be wrapped in an enclosing tag

Correct JSX

```
function App() {  
  return (  
    <div>  
      <h1>Hello</h1>  
      <p>React</p>  
    </div>  
  )  
}
```


Fragment

Instead of <div>, we can use Fragment:

```
<>
  <h1>Hello</h1>
  <p>React</p>
</>
```

Advantage

- Avoids unnecessary extra HTML tags.

2. Close All Tags

All JSX tags must be properly closed.

Wrong

```

<li>Apple
<input type="submit">
```

Error

JSX syntax error

Correct

```

<li>Apple</li>
<input type="submit" />
```

3. Use camelCase Attributes

JSX attributes use camelCase because JSX converts into JavaScript objects.

HTML	JSX	Reason
class	className	class is reserved keyword in JavaScript
onclick	onClick	JSX uses camelCase for events
onchange	onChange	JSX uses camelCase
readonly	readOnly	JSX uses camelCase

Wrong

```
<h1 class="title">
```

Correct

```
<h1 className="title">
```

4. Passing JavaScript Expressions

Use {} to write JavaScript inside JSX.

Example

```
const name = "React"
<h1>{name}</h1>
```

Output:

React

JSX Expression	React Understands As	Output
{n}	Evaluate variable n	lju
"{n}"	Text " + value of n + text "	"lju"
"{n}"	JavaScript string "{n}"	{n}
'{n}'	JavaScript string '{n}'	{n}
{n}	JavaScript string "n"	n
{n + " test"}	JavaScript expression	lju test
{n.toUpperCase()}	Execute JS method	LJU
{5 + 5}	JavaScript math expression	10
{[1,2,3]}	Array rendering	123
{[1,2,3].join(",")}	Array join method	1,2,3

5. JSX Styling

Inline CSS in JSX is written using JavaScript objects.

Correct Styling

```
<h1 style={{color:'red'}}>
  Hello
</h1>
```

Styling Rules

Rule	Example
Use double curly braces	style={{}}
Use camelCase CSS	backgroundColor
CSS values are strings	"red"

Wrong

```
<h1 style="color:red">
```

Error

- JSX expects JavaScript object, not string.

6. JSX Comments

To write comments in React (JSX), we need to wrap **them in curly braces**.

```
Function comment() {  
  Return(  
    { /* this works */ }  
  )  
}
```

The curly braces tell the JSX parser to parse the code inside as JavaScript, and not a string.

Since the contents inside are parsed as JavaScript, this enables us to also do multi-line or single-line comments:

```
function comment(){  
  return (  
    <>  
    {  
      /*  
        mult-line  
        test  
      */  
    }  
    {  
      // single-line test  
    }  
    </>  
  )  
}}
```

In the case of a single-line comment, You cannot have the ending bracket in the same line, because that will break everything. **{// comments }-not valid**

Example to understand jsx rules

Myapp.jsx

```
import './My.css'
import img1 from './assets/hero.png'

function MyApp() {
  const name = "React"

  return (
    <>
      {/* Inline CSS */}
      <h1 style={{
        color: "blue",
        fontStyle: "italic",
        fontSize: "25px"
      }}>
        Welcome to {name}
      </h1>

      {/* External CSS using className */}
      <p className="para">
        JSX allows writing HTML-like code inside JavaScript.
      </p>

      {/* Ordered List */}
      <h3>List of Fruits</h3>
      <ol type="A">
        <li>Apple</li>
        <li>Mango</li>
        <li>Kiwi</li>
      </ol>

      {/* Image from assets folder */}
      <img src={img1} alt="fruit" width="200" />

      {/* Image from public folder */}
      

    </>
  )
}
```

```
)  
}  
export default Myapp
```

My.css

```
.para {  
  color: green;  
  font-size: 20px;  
}
```

Import `Myapp.jsx` component in `Main.jsx` and replace ``<App />`` with ``<Myapp />`` to render the custom component in browser.

If you wish to see only your applied CSS, then comment out:

- index.css import from main.jsx
- App.css import from App.jsx

Example:

```
// import './index.css'  
// import './App.css'
```

Map and Filter

Purpose of map() in React

- map() is used to loop through array elements.
- It helps to display data dynamically in React.
- It transforms each array item into JSX.
- It returns a new array.
- Mostly used for rendering lists.

Purpose of filter() in React

- filter() is used to remove unwanted data.
- It checks condition on every array element.
- It returns only matching elements.
- It creates a filtered array.
- Mostly used before map().

List and Keys

- ✓ key is used to give a unique identity to each element in a list.
- ✓ React uses key to identify which item is:
 - a. added
 - b. updated
 - c. deleted
- ✓ It helps React update the UI efficiently.
- ✓ key improves performance while rendering lists.
- ✓ It is mainly used with map() method.
- ✓ If **lists** do not include the **key** attribute, then it will give below warning. The warning says that each of the list items in our unordered list should have a unique key.

**Warning: Each child in an array or iterator
should have a unique "key" prop**

Keys are used in React to identify which items in the list are changed, updated, or deleted. In other words, we can say that keys are used to give an identity to the elements in the lists. It is recommended to use a string as a key that uniquely identifies the items in the list.

Use of key is given in below example. Without adding it you will get a warning as mentioned above.

Example

Create a React component that contains an array of student objects with:

- id
- name
- standard (std)

Perform the following tasks using filter() and map():

1. Display only students whose standard is 5.
2. Increase their standard by 1.
3. Display student name and updated standard.

ProductList.jsx

```
function StudentList() {
  const students = [
    {
      id: 1,
      name: "abc",
      std: 4
    },
    {
      id: 2,
      name: "xyz",
      std: 6
    },
    {
      id: 3,
      name: "pqr",
      std: 5
    }
  ];

  return (
    <div>
      <h1>Student Details</h1>

      {
        students
          .filter((student) => student.std === 5)
          .map((student) => (
            <div key={student.id}>
```

```
<h2>Student Name : {student.name.toUpperCase()}</h2>
```

```
<h3>Updated Standard : {student.std + 1}</h3>
```

```
<hr />
```

```
</div>
```

```
)
```

```
}
```

```
</div>
```

```
);
```

```
}
```

```
export default StudentList;
```


React Props

- ✓ Props stand for **Properties**.
- ✓ Props are used to pass data from one component to another component.
- ✓ Props work similar to function arguments in JavaScript.
- ✓ Props help React components communicate with each other.
- ✓ Props are read-only (immutable), meaning we should not modify them directly.

Why We Use Props

Props are used to:

- pass data between components
- make components reusable
- display dynamic content
- communicate from parent component to child component

Syntax of Props

Passing Props

```
<ComponentName propertyName="value" />
```

Reading Props

```
function ComponentName(props) {  
  return <h1>{props.propertyName}</h1>  
}
```

Example

Create a React application using props to display student details.

Requirements:

1. Create a parent component Prop1.js.
2. Pass the following data to child component Prop2.js using props:
 - student name
 - roll number
 - marks
3. In child component:
 - display student details
 - increase marks by 1 and display updated marks

Understand the difference between:
`marks={20}` `marks="22"` from example

Step 1: Pass props to the component

Prop1.jsx

```
import Prop2 from "./Prop2";

function Prop1 () {
  var n = "ABC";
  return (
    <div>
      <Prop2 name={n} rollnum="101" marks={20} />
      <Prop2 name="DEF" rollnum="102" marks={16} />
      <Prop2 name="GHI" rollnum="103" marks="22" />
    </div>
  );
}
export default Prop1;
```

Step 2: Read props inside the component

Prop2.jsx

```
function Prop2(props){
  return(
    <div>
      <ul>
        <li>{props.name}</li>
        <li>{props.rollnum}</li>
        <li>{props.marks + 1}</li>
      </ul>
    </div>
  );
}
export default Prop2;
```

Output:

ABC

101

21

DEF

102

17

GHI

103

221

Difference Between "" and {}

Syntax	Type
"22"	String
{22}	Number / JavaScript value

Example

Create a React application using props to pass an object from one component to another component.

Requirements:

1. Create a parent component Example1.js.
2. Create an object named Details containing:
 - Student_name
 - University_name
3. Pass the Details object to child component Example.js using props.
4. In child component:
 - receive the object using props
 - display student name and university name in browser

Example1.jsx

```
import Example from './Example';  
function Example1 () {  
  const Details = {Student_name: "abc", University_name: "LJU"};  
  return (  
    <div>  
      <Example data ={ Details }/>  
    </div>  
  )  
}
```

```
</div>
)}
export default Example1
```

Example.jsx

```
function Example(props) {
return(
<h2> My name is {props.data.Student_name}.
I am a student of { props.data.University_name } University !</h2>
)}
export default Example
```

Output:

My name is abc. I am a student of LJU University !

Example

Build a React app that displays a list of products using props.

- Create a **ProductCard** component to show product details like title, price, rating, and image.
- Use a **ProductList** component to store product data and display multiple **ProductCard** components using **.map()**.

ProductList.jsx

```
import ProductCard from './ProductCard';
import img1 from "./img1.png"
import img2 from "./img2.png"

function ProductList() {
  const products = [
    {
      title: "iPhone 17 Pro",
      price: "99999",
      rating: 4.8,
      image: img1
    },
    {
      title: "Samsung Galaxy S24",
      price: "60000",
      rating: 4.5,
      image: img2
    }
  ]
}
```

```

    }
  ];

  return (
    <div>
      <h1>Our Products</h1>
      <ProductCard productList={products} />
    </div>
  );
}

```

```
export default ProductList;
```

ProductCard.jsx

```

function ProductCard(props) {
  return (
    <>
      {
        props.productList.map((product, index) => (
          <div>
            <img src={product.image} alt={product.title} width="100%" />
            <h2>{product.title}</h2>
            <p>Price: {product.price}</p>
            <p>Rating: {product.rating}</p>
          </div>
        ))
      }
    </>
  );
}

export default ProductCard;

```

React Events

To handle events with React elements is very similar to handle events with DOM events. There are some syntax differences as below:

React events are named using camelCase, rather than lowercase as we used to do in HTML.

In HTML:

```
<button onclick="demofunction()">
  LJ University
</button>
```

In React:

```
<button onClick = { demoFunction }>
  LJ University
</button>
```

Another difference is that we cannot **return false** to prevent default behavior in React. We must call **preventDefault** explicitly.

preventDefault is used to prevent the default form behavior of submitting.

In HTML

```
<form onsubmit="console.log (' You clicked submit. '); return false">
  <button type="submit">Submit</button>
</form>
```

In React

```
function Form () {
  function handleSubmit (e) {
    e.preventDefault ();
    alert (' You clicked submit. ');
  }

  return (
    <form onSubmit = {handleSubmit}>
      <button type="submit">Submit</button>
```

```
</form>
);
}
```

1) Example of onclick event

Write react js script to display alert box with text “Welcome to LJU” by clicking on button.

Event1.jsx

```
function Event1() {
  const mystyle = {
    color : "white",
    backgroundColor: "#000000",
    padding: "10px 20px",
    margin: "200px"
  };
  function handleClick () {
    alert ('Welcome to LJU');
  }
  return (
    <div>
      <center>
        <button style = {mystyle} onClick={handleClick}>
          Click me
        </button>
      </center>
    </div>
  );
}
export default Event1;
```

App.jsx

```
import Event1 from "../Event1"
function App() {
  return (
    <div>
      <Event1/>
    </div>
  );
}
```

```
}  
export default App;
```

Output:



2) Example of onchange event

Write react js script to display values in console while changing it in text box.

Event2.jsx

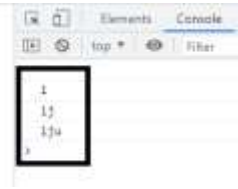
```
import React from 'react';  
function Event2() {  
  function handleChange(event) {  
    console.log (event.target.value);  
  }  
  return (  
    <input type="text" name="firstName" onChange={handleChange} />  
  );  
}  
export default Event2;
```

App.jsx

```
import Event2 from "./Event2"  
function App() {  
  return (  
    <div>  
      <Event2/>  
    </div>  
  );  
}  
export default App;
```

Output:

OnChange Example



`event.target` gives the element that triggered the event.

So, `event.target.value` retrieves the value of that element (an input field, in above example).

3) Example of DoubleClick event

Write react js script to display alert box with text “welcome to lju” only on double click.

Event3.jsx

```
import React from 'react';
function Event3() {
  const mystyle = {
    color : "white",
    backgroundColor : "#000000",
    padding: "10px 20px",
    margin:"30px"
  };

  const doubleClickHandler = (event) => {
    alert("Welcome to LJU");
  }
  return (
    <>
      <button style={mystyle} onDoubleClick = {doubleClickHandler}>Double Click
here</button>
    </>
  );
}
export default Event3;
```

App.jsx

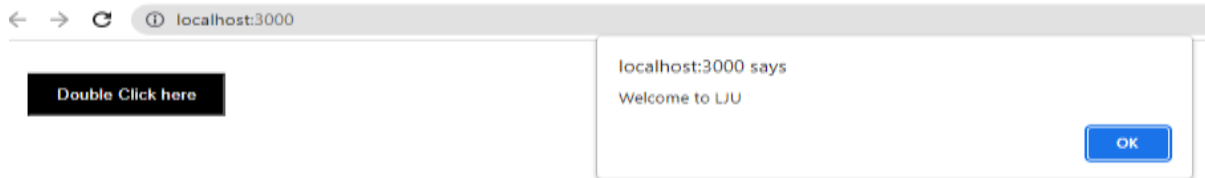
```
import Event3 from "../Event3"
function App() {
  return (
```

```

<div>
  <Event3/>
</div>
);
}
export default App;

```

Output:



4) Example of onSubmit event

Write react js script to display alert box with text “You clicked submit.” only on submitting form.

Event4.jsx

```

function Event4() {
  function handleSubmit (e) {
    e.preventDefault ();
    alert (' You clicked submit. ');
  }

  return (
    <form onSubmit = {handleSubmit}>
      <button type="submit">Submit</button>
    </form>
  );
}export default Event4;

```

App.jsx

```

import Event3 from "./Event4"
function App() {
  return (
    <div>
      <Event4/>
    </div>
  );
}

```

```
}  
export default App;
```

Example to understand concept of function calling with ()

```
function App() {  
  const handleClick = () => {  
    alert('Welcome to LJU');  
  };  
  return (  
    <div>  
      <center>  
        <button onClick={handleclick()}>Click me</button>  
      </center>  
    </div>  
  );  
}
```

export default App;

Output:

On page load, the alert pops up: "Welcome to LJU"

But when you click the button, nothing happens because the onClick handler is now undefined.

Explanation:

The function handleClick() is being called immediately inside onClick={handleclick()} instead of passing a reference.

So, as soon as the component renders, it will call the function handleClick(). An alert box appears with the message "Welcome to LJU"

React Routing

- ✓ React does not include page routing by default.
- ✓ To perform routing in React applications, we use React Router.
- ✓ React Router is the most popular library used for routing in React applications.
- ✓ It helps create Single Page Applications (SPA) where pages change without reloading the browser.

Install React Router

Run the following command in terminal:

```
npm install react-router-dom
```

Purpose of React Router

React Router is used to:

- navigate between pages
- create SPA (Single Page Application)
- change URL dynamically
- render components based on URL path

Important Components in React Router

1. BrowserRouter

BrowserRouter enables routing in React application and keeps UI synchronized with URL. It acts as the parent component for routing.

Syntax

```
<BrowserRouter>  
  App Components  
</BrowserRouter>
```

2. Routes

<Routes> is used to group multiple <Route> components. React checks routes and renders the best matching route.

Syntax

```
<Routes>  
  <Route path="/" element={<Home />} />  
</Routes>
```

3. Route

<Route> is used to define which component should render for a specific URL path.

Syntax

```
<Route path="/home" element={<Home />} />
```

4. Link

<Link> is used for navigation between pages without refreshing the browser. It works similar to HTML anchor tag.

Syntax

```
<Link to="/home">Home</Link>
```

Important Note

Do Not Use

```
<a href="/home">Home</a>
```

because it reloads the page.

Use

```
<Link to="/home">Home</Link>
```

to maintain SPA behavior.

Routing Flow

URL changes



React Router checks matching path



Matching component renders

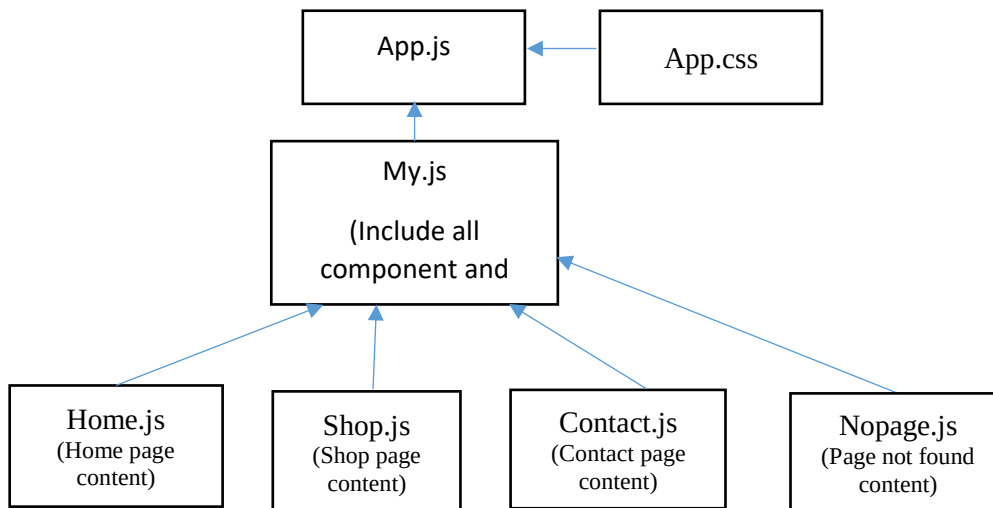
Example

Create a React application using **functional components** and **React Router DOM**.

Requirements

1. Create routes for:
 - Home
 - Shop
 - Contact
 - 404 NoPage
2. Implement navigation using Link.
3. Display:
 - **Home page** on Home route
 - **Shop page** on Shop route
 - **Contact Detail** on Contact route
 - **404 Page not Found** for invalid URL

- Create My.js file which contains Links for Home, Shop and Product page. Also, add functionality of page routing. Finally call My.js in App.js.



App.jsx

```

import './App.css';
import My from "../routing/My.jsx";

function App() {
  return (
    <div>
      <My/>
    </div>
  );
}

export default App;
  
```

(Below all files are inside routing folder in src folder.)

routing/My.jsx

```

import React from 'react';
import {BrowserRouter as Router,Route,Routes,Link} from "react-router-dom";
import Home from './Home';
import Contact from './Contact'
import Shop from './Shop'
import Nopage from './Nopage';
function My() {
  return (
    <div>
  
```

```

<Router>
  <div className='main-route'>
    <ul>
      <li><Link to="/">Home</Link></li>
      <li><Link to="/shop">Shop</Link></li>
      <li><Link to="/contact">Contact</Link></li>
    </ul>
  </div>
  <Routes>
    <Route path="/" element={<Home/>}/>
    <Route path="/contact" element={<Contact/>}/>
    <Route path="/shop" element={<Shop/>}/>
    <Route path="*" element={<Nopage/>}/>
  </Routes>
</Router>
</div>
);
}
export default My

```

Setting the path to * will act as a catch all undefined URLs and display 404 error page.

routing/Home.jsx

```

import React from 'react'
function Home(){
  return (
    <div>
      <h1>Home page</h1>
    </div>
  )
}
export default Home

```

routing/Shop.jsx

```

import React from 'react'
function Shop(){
  return (
    <div>
      <h1>Shop page</h1>
    </div>
  )
}

```

```
)  
}  
export default Shop
```

routing/Contact.jsx

```
import React from 'react'  
function Contact() {  
  return (  
    <div>  
      <h1>Contact Detail</h1>  
    </div>  
  )  
}  
export default Contact
```

routing/Nopage.jsx

```
import React from 'react'  
function Nopage() {  
  return (  
    <div>  
      <h1>404 Page not Found</h1>  
  
    </div>  
  )  
}  
export default Nopage
```

App.css (It is Not compulsory to add this file in this example. Added css for the reference only)

```
.main-route ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
  overflow: hidden;  
  background-color: #000;  
  margin-bottom: 50px;  
}  
.main-route li {  
  float: left;  
}
```



```
.main-route li a {  
  display: block;  
  color: white;  
  text-align: center;  
  padding: 20px;  
  text-decoration: none;  
}  
.main-route h1{ color: red; text-align: center;
```

Miscellaneous Examples

Example

Create a React app to perform the following tasks using functional components:

Implement the following components in your React application:

- Main.js to set up the router and define the routes.
 - Home.js for the Home page.
 - Product.js for the Product page.
1. **Create a React Router:**
 - Include two routes: Home and Product. Implement navigation between these routes.
 2. **Create the following routes and components:**
 - When a user clicks on the Home page link, it should navigate to the Home page and display "Welcome to LJU" within an <h1> heading with blue color. Also, include link to product page.
 - A **Product** page that displays three products' information (name, price, and image) using props. When a user clicks on the Product page link, it should navigate to the Product page and display three products' information name, price and product image using props.

Main.jsx

```
import {BrowserRouter as Router,Route,Routes,Link} from "react-router-dom";
import Home from './home';
import Product from './product'
import Nopage from './nopage';
import img1 from "./img1.jpg"
import img2 from "./img2.jpg"
import img3 from "./img3.png"
function Main() {
  const products=[{name:"p1",price:20000, pic:img1},{name:"p2",price:14000,
pic:img2},{name:"p3",price:40000, pic:img3}]
  return (
<div>
  <Router>
    <ul>
      <li><Link to="/">Home</Link></li>
      <li><Link to="/product">Product</Link></li>
    </ul>
```

```

    <Routes>
      <Route path="/" element={<Home/>}/>
      <Route path="/product" element={<Product info={products}/>}/>
      <Route path="*" element={<Nopage/>}/>
    </Routes>
  </Router>

</div>
);
}
export default Main

```

home.jsx

```

function Home(){
  return (
    <div>
      <h1 style={{color:"blue"}}>Home page</h1>
      <a href="/product">Product</a>
    </div>
  )
}
export default Home

```

nopage.jsx

```

function Nopage() {
  return (
    <div>
      <h1>404 Page not Found</h1>
    </div>
  )
}
export default Nopage

```

product.jsx

```

const Product = (Props)=>{
  return(
    <div>
      {
        Props.info.map((p)=>{
          return(
            <div>

```

```

        <img src={p.pic} alt="product" height={200} width={200}/>
        <h1>{p.name}</h1>
        <h1>{p.price}</h1>
      </div>
    )
  })
}
</div>
);
}
export default Product;

```

Example

Create a component to perform the tasks as described below:

1. Add a text field and a submit button.

- While changing the value in the text field, display it below the form.
- Display this text field value in an alert box upon submitting it.

2. Add a button to perform click and double-click event tasks.

- On click event, display message in h3 tag “You clicked once”.
- On double-click event, display message in h3 tag “You clicked twice”.
- Message should be displayed below the button.

```

function Map1() {
  const arr=[1,2,3,4,5];
  function handleSubmit (e) {
    e.preventDefault ();
    alert (document.getElementById('uname').value);
  }
  function handleClick(){
    document.getElementById('test1').innerHTML = “You clicked once”
  }
  function handledoubleclick(){
    document.getElementById('test1').innerHTML = “You clicked twice”
  }
  function handleChange(event) {
    document.getElementById('test').innerHTML =event.target.value;
  }
}

```

```
return (  
  <div>  
    <form onSubmit = {handleSubmit}>  
      <input type="text" id="uname" onChange={handleChange}></input>  
      <input type="submit"/>  
    </form>  
  
    <h1 id="test">On change event</h1>  
  
    <button style={{backgroundColor:'black',padding:"20px",color:"white"}}  
onClick={handleclick} onDoubleClick={handledoubleclick}>Click</button>  
    <h1 id="test1">Click/DoubleClick event</h1>  
  </div>  
)  
}  
  
export default Map1
```

Reference

public/ vs src/assets/

public/	src/assets/
Static files	Component-related files
Direct URL access	Imported into React components
No Vite optimization	Optimized by Vite
Good for favicon, PDFs	Good for images/icons used in components

Example

Store in public/

public/logo.png

Use directly:

```

```

Store in src/assets/

src/assets/logo.png

Import before use:

```
import logo from './assets/logo.png'  
<img src={logo} />
```